College of Computing

Lab: Inheritance & Record & String
Professor : Fahd KALLOUBI
Year : 2026/2027
Major : Computer Science (CS1)

In this Lab, we will dive into inheritance concepts in java alongside some important concepts related to class/object creation. In addition, we will manipulate I/O streams in java and the String/StringBuilder classes.

Learning outcomes:

- Understand encapsulation
- Manipulate inheritance and the super(), this() keywords
- Master string manipulation and advanced formatting
- Translate basic class diagrams into java classes

Problem 1:

Start by forking and cloning the repository “problem1-lab4” locally.

File Sales.java (package problem1) contains a Java program that prompts for and reads in the sales for each of 5 salespeople in a company. It then prints out the id and amount of sales for each salesperson and the total sales. Study the code, then compile and run the program to see how it works. Now modify the program as follows:

1. Compute and print the average sale. (You can compute this directly from the total; no loop is necessary.)
2. Find and print the maximum sale. Print both the id of the salesperson with the max sale and the amount of the sale, e.g., "Salesperson 3 had the highest sale with $4500." Note that you don't need another loop for this; you can do it in the same loop where the values are read and the sum is computed.
3. Do the same for the minimum sale.
4. After the list, sum, average, max and min have been printed, ask the user to enter a value. Then print the id of each salesperson who exceeded that amount, and the amount of their sales. Also print the total number of salespeople whose sales exceeded the value entered.
5. The salespeople are objecting to having an id of 0—no one wants that designation. Modify your program so that the ids run from 1–5 instead of 0–4. Do not modify the array—just make the information for salesperson 1 reside in array location 0, and so on.
6. Instead of always reading in 5 sales amounts, at the beginning ask the user for the number of sales people and then create an array that is just the right size. The program can then proceed as before.

Problem 2:

File IntegerList.java, in package problem2, contains a Java class representing a list of integers. The following public methods are provided:

- IntegerList(int size)—creates a new list of size elements. Elements are initialized to 0.
- void randomize()—fills the list with random integers between 1 and 100, inclusive.
- void print()—prints the array elements and indices

File IntegerListTest.java contains a Java program that provides menu-driven testing for the IntegerList class. Compile and run IntegerListTest to see how it works.

It is often necessary to add items to or remove items from a list. When the list is stored in an array, one way to do this is to create a new array of the appropriate size each time the number of elements changes, and copy the values over from the old array. However, this is rather inefficient. A more common strategy is to choose an initial size for the array and add elements until it is full, then double its size and continue adding elements until it is full, and so on. (It is also possible to decrease the size of the array if it falls under, say, half full, but we won't do that here).

1. Add this capability to the IntegerList class. You will need to add an increaseSize method plus instance variables to hold the current number of integers in the list and the current size of the array. Since you do not have any way to add elements to the list, you won't need to call increaseSize yet.

2. Add a method void addElement(int newVal) to the IntegerList class that adds an element to the list. At the beginning of addElement, check to see if the array is full. If so, call increaseSize before you do anything else.

   Add an option to the menu in IntegerListTest to test your new method.

3. Add a method void removeFirst(int newVal) to the IntegerList class that removes the first occurrence of a value from the list. If the value does not appear in the list, it should do nothing (but it's not an error). Removing an item should not change the size of the array, but note that the array values do need to remain contiguous, so when you remove a value you will have to shift everything after it down to fill up its space. Also remember to decrement the variable that keeps track of the number of elements.

   Add an option to the menu in IntegerListTest to test your new method

4. Add a method removeAll(int newVal) to the IntegerList class that removes all occurrences of a value from the list. If the value does not appear in the list, it should do nothing (but it's not an error).

   Add an option to the menu in IntegerListTest to test your new method

Problem 3:

Before starting this problem, try to compile and run Circle.java in the package “recordemo” to see how it works.

Create a WeatherData (already created in the same package) record to store temperature and conditions. Add methods for temperature unit conversion and generating a formatted summary string.

1. Define a WeatherData record with components double temperatureCelsius and String conditions.
2. Add an instance method `double temperatureFahrenheit()` that converts the temperature from Celsius to Fahrenheit (C*9/5+32).

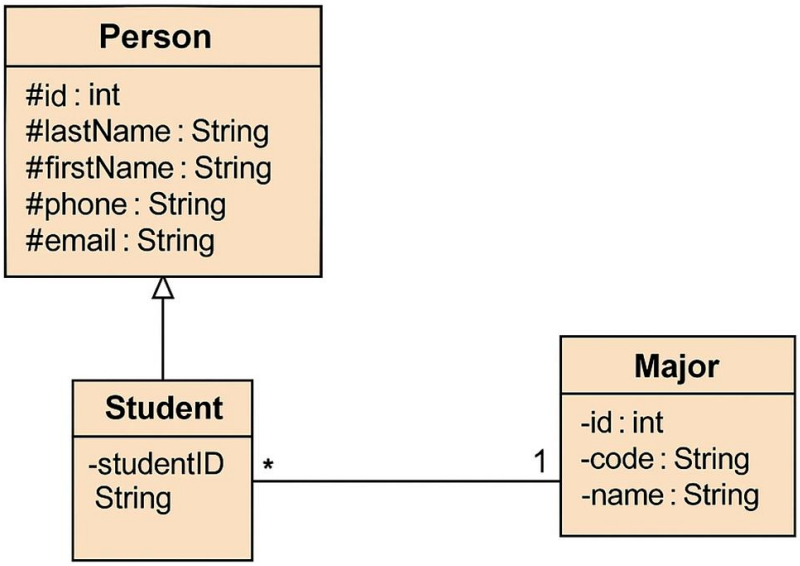
3. Add an instance method String getSummary() that returns a formatted string with both temperatures and the conditions, like: "Current weather: 25.0°C (77.0°F) and Sunny". (try to use the method format of the String class)
4. Add a static factory method WeatherData fromFahrenheit(double tempFahrenheit, String conditions) that takes a Fahrenheit temperature and creates a WeatherData record by converting the temperature to Celsius first.
5. In a main method, demonstrate both the instance methods and the static factory method.

Expected Output:

```
Today's weather: Current weather: 25.0°C (77.0°F) and Sunny
Yesterday's weather: Current weather: 10.0°C (50.0°F) and Cloudy
```

Problem 4:

Given the following class diagram:



Develop the above classes inside the package “student”.

Each class must include:

- A no arguments constructor
- An initialization constructor
- Accessor methods (getters and setters)
- The toString() method.

Note: The identifier (id) must be auto-incremented.

1. The default major is computer science (code: 23), implement a constructor to create computer science students without specifying the major (think about constructor chaining)
2. A major cannot exceed 50 students

3. When a student is created, he should be automatically linked to his corresponding major.
4. In the Test class:
    a. Create a couple of majors
    b. Create some students associated with these majors
    c. Display computer science students

Output:

```
The list of students in the computer science major is:
  1. 22885676  SAFI Amal
  2. 23585976  ALAMI Samir
```

5. In the Student class, create a method getFullNameFormatted(), that returns the name in the format "LASTNAME, Firstname" using String.format() or StringBuilder (LASTNAME is on uppercase)
6. Modify the Major class to include the method: findStudentByCNE(String cne), This method should search for and return the student whose CNE matches the given value, or null if not found.
7. Add a method in the Major class to count how many students are enrolled: getStudentCount()
8. Write a method in Major to remove a student by CNE: removeStudent(String cne), it should call the findStudentByCNE(String cne), The method should return true if the student was found and removed, otherwise false.
9. Add a method that displays the percentage of occupied capacity: getOccupancyRate().

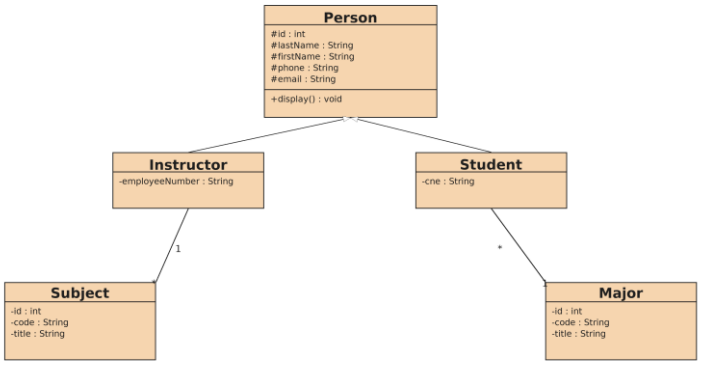
    Example:

```
Computer science capacity: 50 students
Current enrollment: 25 students
Occupancy rate = 50.0%
```

10. write a method: getStudentListAsString(), that returns a single formatted string containing all students, separated by line breaks, using StringBuilder instead of string concatenation.
11. In the Test class, call the newly created methods.

Problem 5:

I have extended the previous class diagram by adding the instructor and the subject classes.



In the package instructor, create the Instructor and Subject classes.

Try to add the following methods:

1. Write a method String cleanEmployeeNumber() in Instructor that trims whitespace and removes internal spaces from employeeNumber.
   Example: " AB 123 " → "AB123".
2. In Subject, implement String normalizedCode() that returns code upper-cased and trimmed.
   Example: " cs-101 " → "CS-101".
3. Add String properTitle() to Subject that capitalizes the first letter of each word in title (simple split on spaces; ignore complex locales).
   Example: "introduction to java" → "Introduction To Java".
4. Implement String summaryLine() in Instructor using String.format to return: "Instructor[employeeNumber=%s, lastName=%s, firstName=%s]".
5. In Subject, add boolean isIntroCourse() that returns true if title contains "Intro" (case-insensitive) or code starts with "INTRO-".
6. Implement String toCard() in Instructor using StringBuilder that returns a multi-line block:

   ```
   Instructor
   ----------
   Employee #: <employeeNumber>
   Name      : <lastName>, <firstName>
   Email     : <email>
   Phone     : <phone>
   ```

7. In Subject, write String syllabusLine() using StringBuilder to produce:
   "<code> - <title> (Instructor: <lastName> <firstName>)".
   Assume you receive an Instructor parameter.
8. In Instructor, implement String displayName() that handles nulls safely (e.g., if firstName is null, show only lastName) using StringBuilder and conditional appends.

Problem 6:

In package problem6, write the classes necessary for the following program to run.

```
   public class TestFormes {
   public static void main(String[] argv) {
       Forme[ ] figures = new Forme[3] ;
       figures [ 0 ] = new Square( 2 ) ;
       figures [ 1 ] = new Circle( 3 ) ;
       figures [ 2 ] = new Square( 5.2 ) ;
       for( int i=0 ; i< figures.length ; i++ )
             System.out.println( figures[i] +" : area = "+
figures[i].getSurface() +"
   cm2" ) ;
   }
   }
```

Output:

```
Square (side 2.0 cm): area = 4.0 cm2
Circle (radius 3.0 cm): area = 28.26 cm2
Square (side 5.2 cm): area = 27.04 cm2
```

Problem 7:

In package problem6, write the classes necessary for the following program to run.

```
   public class TestMetiers {
       public static void main(String[] argv) {
             Person[ ] people = new Person[3] ;
             people [ 0 ] = new Carpenter( "Paul" ) ;
             people [ 1 ] = new Plumber( "Jean" ) ;
             people [ 2 ] = new Carpenter( "Adrien" ) ;
             for( int i=0 ; i< people.length ; i++ )
people[i].display() ;
   }
   }
```

Output:

```
I am Paul the Carpenter
I am Jean the Plumber
I am Adrien the Carpenter
```